

Assignment 2

[GITHUB LINK](#)

FLUTTER ASSIGNMENT REPORT

TITLE:

Working with Mock API, Asynchronous Data Fetching and Error Handling in Flutter

=====

1. INTRODUCTION

=====

This assignment was developed using Flutter and Dart to understand how a Flutter application can work with API-based data.

In real-world applications, data is usually retrieved from a backend server rather than being permanently written inside the user interface. Since this assignment does not require a real backend server, a Mock API was created to simulate API behavior.

The main purpose of the assignment was to understand the complete process of fetching data asynchronously and displaying it in the Flutter UI. The application also demonstrates how to handle different situations such as loading, API failure, retrying a request, null data, empty data, and successful data.

The project is divided into three main files:

1. product.dart – Contains the Product data model.
2. mock_api.dart – Simulates API requests and responses.
3. main.dart – Contains the Flutter UI and API data-handling logic.

The Product class contains an ID, name, and optional price. The price is nullable because API data may sometimes not contain a value.

=====

2. OBJECTIVES

The main objectives of this assignment were:

- To understand the basic concept of API communication in Flutter.
- To create a model class for API data.
- To understand asynchronous programming using Future.
- To use async and await for asynchronous operations.
- To simulate network delay and API failure.
- To handle loading states.
- To handle API exceptions.
- To provide a retry mechanism.
- To handle null and empty responses.

- To display API data dynamically using ListView.builder.
- To understand StatefulWidget and setState().
- To connect application data with the Flutter user interface.

=====

3. PRODUCT DATA MODEL

The Product class is used to represent product information.

It contains three properties:

- id – identifies the product.
- name – stores the product name.
- price – stores the product price.

The price property is declared as:

double? price

The question mark means that price is nullable. Therefore, a product can have a price or the price can be unavailable.

For example, the application can represent:

Product(id: 1, name: 'Pizza', price: 299)

and also:

Product(id: 3, name: 'Pasta', price: null)

This demonstrates how an application can safely represent optional API data.

The Product model uses required values for id and name while allowing price to remain optional.

=====

4. MOCK API

The MockApi class is used to simulate a real API.

The fetchProducts() method returns product data asynchronously. A two-second delay is introduced using Future.delayed() to simulate the time normally required for a network request.

The Mock API also intentionally generates an error during the first API request. A Boolean variable named _firstRequest is used to track this behavior.

During the first request, an exception is thrown. After that, the value is changed so that another request can succeed.

The Mock API can also return null when requested. Finally, the successful response returns three products:

- Pizza – ₹299
- Burger – ₹199
- Pasta – Price unavailable

The Pasta product has a null price, which allows the application to demonstrate nullable API data.

=====

5. FLUTTER APPLICATION AND STATE MANAGEMENT

The main.dart file contains the main Flutter application.

The application starts from the main() function and runs MyApp. MaterialApp is used to configure the application, and ProductPage is set as the home screen.

ProductPage is a StatefulWidget because its UI changes according to the current API state.

The page maintains three important state variables:

List? products;

This stores the products returned by the API.

String? errorMessage;

This stores an error message when an API request fails.

bool isLoading = true;

This indicates whether data is currently being loaded.

These variables allow the application to display the correct interface according to the current state.

=====

6. ASYNCHRONOUS DATA FETCHING

The initial API request is started from initState() by calling fetchData().

The fetchData() method is asynchronous and uses async/await.

When the method starts, the application sets isLoading to true and clears any previous error message.

The API request is then performed using:

await api.fetchProducts();

The await keyword allows the application to wait for the API response without blocking the normal Flutter application flow.

After a successful response, the returned product list is stored in products, and isLoading is changed to false.

The general process is:

```
Start loading
↓
Call API
↓
Wait for response
↓
Receive product data
↓
Update state
↓
Display products
```

This is a common pattern for Flutter applications that communicate with APIs.

=====

7. LOADING AND ERROR HANDLING

The application provides a loading state while waiting for the API response.

If isLoading is true, a CircularProgressIndicator is displayed in the center of the screen.

This gives the user feedback that the application is currently processing the request.

The API request is also placed inside a try-catch block.

If the request fails, the catch block sets the error message to:

"Failed to fetch products"

and changes isLoading to false.

Instead of allowing the application to crash, the UI displays an error message and a Retry button.

The Retry button calls fetchData() again, allowing the user to make another API request.

This is useful in real applications because network requests can fail temporarily and users should have an opportunity to try again.

=====

8. NULL, EMPTY AND SUCCESS STATES

The application also handles situations where there are no products.

If products is null or the list is empty, the application displays:

"No products available"

This is different from an API error. An error means that the request failed, while an empty result means that there is no product data to display.

When products are successfully received, `ListView.builder` is used to dynamically create the product list.

Each product is displayed inside a Card and ListTile.

The product ID is displayed inside a CircleAvatar.

The product name is displayed as the title.

The price is displayed as the subtitle.

If the price is null, the application displays:

"Price not available"

Otherwise, it displays the actual price.

This makes the interface dynamic because the widgets are generated from the product data instead of being manually written for each product.

=====

9. COMPLETE APPLICATION FLOW

The complete working flow of the application is:

1. The Flutter application starts.
2. `ProductPage` is opened.
3. `initState()` calls `fetchData()`.
4. The loading state is activated.
5. Mock API waits for two seconds.
6. The first request intentionally fails.
7. The error screen is displayed.
8. The user presses `Retry`.
9. `fetchData()` is called again.
10. The Mock API returns product data.
11. The loading state is removed.
12. The products are displayed using `ListView.builder`.
13. Products with prices display their prices.
14. Products without prices display "Price not available".

This demonstrates the complete lifecycle of an API-driven Flutter screen.

10. CONCLUSION

This assignment provided practical knowledge about handling API-based data in Flutter.

The project demonstrated how to create a data model, simulate an API, perform asynchronous operations, manage loading states, handle exceptions, provide a retry option, handle null and empty responses, and dynamically display data.

One of the most important concepts demonstrated by the assignment is that an application should not only handle the successful API response. It should also properly handle loading, error, null, and empty states.

The use of `StatefulWidget` and `setState()` allows the interface to update when the API state changes. The use of `ListView.builder` makes it possible to display a dynamic number of products.

Overall, this assignment provides a strong foundation for working with real APIs in Flutter. The concepts learned here can later be extended to REST APIs, JSON parsing, authentication, database integration, pagination, and more advanced state-management techniques.